Final Lab – The Orb

Review of Algorithms

Geometry Representation and Processing

The project produces standardised *Geometry* objects, containing indexed vertex buffers stored in array of structures format, defined in *Vertex.h*, containing Position, colour, Texture Coordinates and Normal.

Meshes can be imported as .obj files, where an unordered map is used to map unique combinations of positon/uv/normal indices to a single vertex index in the final buffer. The loader uses hashing to identify unique combinations of attributes, rather than unique positions, meaning that if two faces share a vertex with identical attributes, the vertex can be reused instead of duplicated, reducing memory usage.

The *GeometryGenerator* creates terrain using Perlin noise, using one frequency to define the base layer, giving a general shape, and another to add surface detail. To create island-like geometry, the heights are clamped to zero when the distance from the centre exceeds 95% of the radius, ensuring the terrain seamlessly merges with the top of the pedestal.

When *RecordCommandBuffer* iterates through the scene objects, it updates a *PushConstantObject* struct for every entity. The model matrix is passed via push constant, allowing the same *Geometry* buffer to be drawn multiple at positions without modifying the vertex buffer itself. Control flags like *shadingMode*, *layerMask*, and *burnFactor* are packed into the push constant block. This allows the shader to switch between Gouraud/Phong shading or apply burning effects per-object instantly, avoiding the overhead of descriptor set updates.

Shading and Lighting

The rendering engine implements a hybrid shading pipeline that supports both Gouraud and Phong shading models, with lighting calculations utilising the Blinn-Phong reflection model, where specular highlights are determined by the dot product of the normal and halfway vector. Point lights use quadratic falloff to localise the illumination, while directional lights apply no attenuation.

Refraction is achieved by sampling a captured off-screen framebuffer, where texture coordinated are modified based on the surface normal to simulate distortion. Layer masks are used to restrict light sources to specific geometry, preventing indoor illumination for affecting exterior surfaces.

Shadow Generation

Shadows are generated using a directional shadow mapping technique with an orthographic projection centred on the scene. To mitigate aliasing artifacts along shadow edges, the fragment shader implements percentage-closer filtering, which samples a 3x3 grid of texels around the current fragment and averages the result. Surface acne is precented using a dynamic slope-scaled bias, which adjusts the shadow based on the angle of the light source relative to the surface normal.

Particle System

Hardware instancing is utilised to efficiently render large numbers of particles with a single draw call. A shared quad mesh is stored in the vertex buffer, while per-instance data is supplied by a secondary vertex buffer. The simulation logic, including velocity integration and lifecycle management, is performed on the CPU. The updated instance data is then copied and mapped to a GPU buffer every frame, allowing for dynamic behaviour like size variation and colour interpolation.

Connecting Application Objects and Graphics

The *SceneObject* struct acts as a lightweight container for specific instance data, holding the logical state required to simulate the entity in the world. This includes a transform matrix for position, rotation, and scale; simulation variables such as *currentTemp*, *ObjectState*, and *isFlammable*; and instance-specific rendering instructions like *texturePath*, *shadingMode*, and *burnFactor*. The heavy graphical data is encapsulated in the *Geometry* class, which owns the vertex buffers and indices on the GPU. The link between these systems is established through a *shared_ptr<Geometry>* member within the *SceneObject*, allowing multiple logical objects to reference the same underlying mesh data.

Application behaviour is simulated entirely on the CPU within the *Scene::Update* loop, ensuring strict separation between game logic and rendering. Orbital mechanics are calculated frame-by-frame using quaternion rotation to update the transform matrix, while a thermodynamics system drives state transitions, such as modifying *currentTemp* based on heat sources or incrementing the *burnFactor* as an object chars. For quick visual changes, the engine leverages the decoupling of geometry to swap the *shared_ptr<Geometry>* to an *AshPile* prototype when an object transitions to the *BURNT* state, instantly altering its mesh without destroying the logical entity.

These CPU-side updates are propagated to the graphics pipeline using two primary methods. Continuous state changes, such as the updated transform matrix and scalar *burnFactor*, are packed into a *PushConstantObject* and sent directly to the command buffer via *vkCmdPushConstants* for every draw call, ensuring the shader receives the exact frame state. The particle system utilises memory mapping, where the simulation calculates new particle positions on the CPU and uses *memcpy* to transfer the data to a mapped GPU instance buffer immediately before drawing.

Extensions and Improvements

The main critique of this project is the lack of abstraction of game logic from the *Scene* class and *SceneObject* struct. Further decoupling the transform, rendering, physics, thermodynamics and orbital components would improve the program's modularity, and should be treating as a priority for the next stages. More use could be made of the GPU, with the use of a deferred rendering pipeline for light calculation, instead of a single shader containing multiple if statements, which introduces improved performance and the possibility of new advanced features, such as illuminating sparks. Similarly, a Compute shader could be utilised to alleviate the stress created by particle calculations and the movement of data from the CPU to GPU. The use of a cascaded shadow map would reduce VRAM usage, while providing better detail nearer the camera.

Upon decoupling the game logic from *Scene*, continuations of this project could integrate bump and displacement mapping for improved visuals, along with the support for high dynamic range rendering. The environment could be improved with the inclusion wind, which would impact dust, fire spread and weather. An audio engine would also make an excellent addition to the program.